

Task1

```
package org.example;
import software.amazon.awssdk.auth.credentials.AwsBasicCredentials;
import software.amazon.awssdk.auth.credentials.StaticCredentialsProvider;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.*;
import java.net.URI;

//create table in DynamoDB
public class CreateTable {
    public static void main(String[] args) throws Exception {
        System.out.println("hello create table in DynamoDB");
        AwsBasicCredentials awsCreds = AwsBasicCredentials.create("fakeAccesskey", "secretAccesskey");

        DynamoDbClient client = DynamoDbClient.builder()
            .endpointOverride(URI.create("http://localhost:8000"))
            .region(Region.AP_SOUTH_1)
            .credentialsProvider(StaticCredentialsProvider.create(awsCreds))
            .build();

        String tableName = "Kantipudi_Table";

        try {
            CreateTableRequest request = CreateTableRequest.builder()
                .tableName(tableName)
                .keySchema(KeySchemaElement.builder()
                    .attributeName("ID")
                    .keyType(KeyType.HASH)
                    .build())
                .attributeDefinitions(AttributeDefinition.builder()
                    .attributeName("ID")
                    .attributeType(ScalarAttributeType.N)
                    .build())
                .provisionedThroughput(ProvisionedThroughput.builder()
                    .readCapacityUnits(5L)
                    .writeCapacityUnits(5L)
                    .build())
                .build();
            client.createTable(request);
            System.out.println(tableName + " table is created. ");
        } catch (Exception ex) { // (ResourceInUseException ex) {
            System.out.println("plz choose different tablename as it already exists");
        }
        client.close();
    }
}
```

Application Edit View Window Help

aws NoSQL Workbench

AWS database catalog

Amazon DynamoDB

Data modeler

Visualizer

Operation builder

Documentation

Share feedback

Dark mode

On DDB local

Operation builder

Connection

Name

Built-in DynamoD...

Tables

Search table

Harika_Table

entries

orders

Saved operations

Search operations

Scan Query PartiQL editor More operations

Would you like to give your feedback? We would love to hear your experience in NoSQL workbench.

Attribute name = Type Attribute value More options Scan

[TABLE] - entries Metadata Actions

	hk	harika	ramya	ayaz	bro
☐	r=29Yt8j	\r^vSEUzZs	5230196030439424	fICGMg<b\	P7VCVzUIPx
☐	*7)D@*F*2B	vnl)msc8\	6751867134541824	QPU&.@9+aU	A\$mIU.}D
☐	Uu[dil<K	@S+0OAR>>e	3025144131878912	do[*RNky>	djuTn{V\$u8
☐	lDkdQ/^k,	tXAs{r5	6787118238007296	XCdHk{#F/5	TbA[/9VX<m
☐	lad(>g#`j	AvwV9TOQ@	1225388797722624	e`4hc6+GzX	m.cIV5A5IH

Displaying 5 items Consumed 0.5 RCUs

Task2

```
package org.example;

import software.amazon.awssdk.auth.credentials.AwsBasicCredentials;
import software.amazon.awssdk.auth.credentials.StaticCredentialsProvider;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.AttributeValue;
import software.amazon.awssdk.services.dynamodb.model.ScanRequest;
import software.amazon.awssdk.services.dynamodb.model.ScanResponse;

import java.net.URI;
import java.util.Map;

public class ScanTable {
    public static void main(String[] args) {

        // AWS credentials (fake for local DynamoDB)
        AwsBasicCredentials awsCreds = AwsBasicCredentials.create(
            accessKeyId: "fakeaccess",
            secretAccessKey: "fakesecret"

        );

        // Create DynamoDB client
        DynamoDbClient client = DynamoDbClient.builder()
            .endpointOverride(URI.create("http://localhost:8000"))
            .region(Region.AP_SOUTH_1)
            .credentialsProvider(StaticCredentialsProvider.create(awsCreds))
            .build();

    }
}
```

```

ScanTable.java  DaxDemo.java  DynamoDBAccess.java  Employee.json  LoadingData
14  public class ScanTable {
15      public static void main(String[] args) {
16
17          ScanResponse response = client.scan(scanRequest);
18
19          System.out.println("✅ Connected to DynamoDB Local");
20          System.out.println("\n📋 Employees in Table:");
21
22          // Loop through all items safely
23          for (Map<String, AttributeValue> item : response.items()) {
24              String id = item.get("ID") != null ? item.get("ID").n() : "N/A";
25              String name = item.get("Name") != null ? item.get("Name").s() : "N/A";
26              String address = item.get("Address") != null ? item.get("Address").s() : "N/A";
27
28              System.out.println("ID=" + id + ", Name=" + name + ", Address=" + address);
29          }
30
31          client.close();
32      }
33  }

```

```

package org.example;
import com.fasterxml.jackson.databind.JsonNode;
import com.fasterxml.jackson.databind.ObjectMapper;
import software.amazon.awssdk.auth.credentials.AwsBasicCredentials;
import software.amazon.awssdk.auth.credentials.StaticCredentialsProvider;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.AttributeValue;
import software.amazon.awssdk.services.dynamodb.model.PutItemRequest;

import java.io.InputStream;
import java.net.URI;
import java.util.HashMap;
import java.util.Iterator;
import java.util.Map;

// loading data to our DynamoDB table
public class LoadingData02 {
    public static void main(String[] args) throws Exception {
        // using the AWS credentials
        AwsBasicCredentials awsCreds = AwsBasicCredentials.create("accessKeyId: "fakeaccess", "secretA

        // create a DynamoDB client
        DynamoDbClient client = DynamoDbClient.builder()

```

```

public static void main(String[] args) throws Exception {
    InputStream stream = getClass().getResourceAsStream("Employee.json");
    .getResourceAsStream("Employee.json"));
    System.out.println("the json file in the input stream");
    JsonNode node = mapper.readTree(stream);

    Iterator<JsonNode> iterator = node.elements();

    while (iterator.hasNext()) {
        JsonNode jsonnode2 = iterator.next();
        Map<String, AttributeValue> item = new HashMap<>();
        item.put("ID", AttributeValue.builder().n(jsonnode2.get("ID").asText()).build());
        item.put("Name", AttributeValue.builder().s(jsonnode2.get("Name").asText()).build());
        item.put("Address", AttributeValue.builder().s(jsonnode2.get("Address").asText()).build());

        PutItemRequest request = PutItemRequest.builder()
            .tableName(tableName)
            .item(item)
            .build();

        client.putItem(request);
        System.out.println(" loading the data to the table");
    }
    client.close();
}

```

```
[
  {
    "ID": 1001,
    "Name": "Prasunamba",
    "Address": "India"
  },
  {
    "ID": 1002,
    "Name": "Meher",
    "Address": "Australia"
  },
  {
    "ID": 1003,
    "Name": "K",
    "Address": "USA"
  }
]
```